COS 214 Practical 1

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

- Date Issued: **27 July 2026**
- Date Due: **2 August 2026** at **23:59**
- Submission Procedure: **Upload an archive to FitchFork**
- Marks: **250**

1 Introduction

1.1 Objectives

In this practical you will:

- revise C++ inheritance, polymorphism and dynamic memory management;
- implement the **Factory Method** pattern to create objects without naming concrete classes;
- implement the **Prototype** pattern to clone pre-configured objects;
- implement the **Template Method** pattern to fix an algorithm's skeleton; and
- implement the **Memento** pattern to capture and restore object state.

1.2 Outcomes

When you have completed this practical you should be able to:

- identify the participants of each of the four patterns in a design;
- implement each pattern in C++11 with correct ownership and no memory leaks; and
- explain the design trade-offs each pattern addresses.

2 Constraints

1. This assignment must be completed **in pairs** (groups of two). Both partners submit, and both names must appear in the PDF.
2. You may use any C++11 compliant compiler; compile with `-std=c++11`.
3. You may include **only** the libraries `vector`, `string`, `iostream` and `map`.
4. You may ask the Teaching Assistants for help, but they will not give you the solutions.
5. Your code is tested for memory leaks. Validate all input to functions.

3 Submission Instructions

Upload all your source files (`.h` and `.cpp`), your `Makefile`, your `main.cpp` and a single PDF of written answers, in one archive, to the appropriate slot on FitchFork before the deadline. Zip **the files**, not the folder containing them. No extension will be granted.

4 Mark Allocation

Task	**Marks**
Source connectors	55
Transformations	55
The pipeline lifecycle	60
Checkpointing and recovery	55
Putting it all together	25
TOTAL	250

5 Scenario

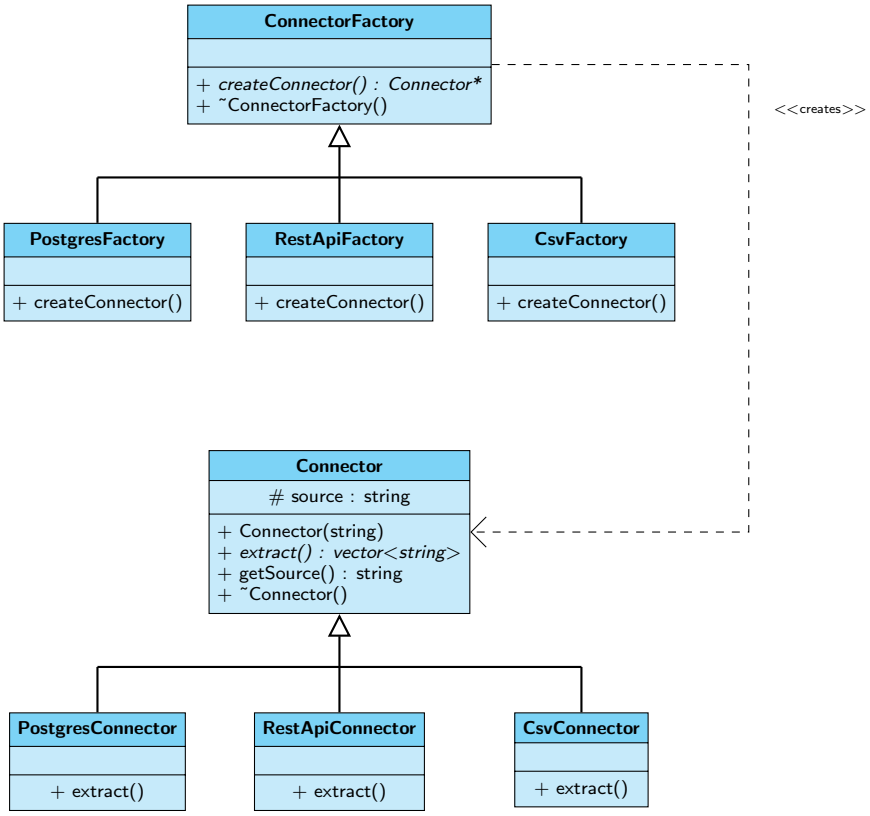
You have been contracted by **Kudu Freight**, a national courier company, to build the core of a nightly *data pipeline engine*. Every night the company ingests delivery records from three very different sources — a warehouse SQL database, a courier partner's REST API, and legacy CSV exports from an old depot system — then cleans, deduplicates and aggregates them before loading them into the reporting warehouse that management reads the next morning.

Two realities shape the design. New data sources are onboarded regularly, so the engine must support a new source **without changing existing code**. Runs are large and occasionally fail at 02:00, so the engine must **resume from the last completed stage** rather than restart. You must implement each of the four patterns below in the part of the system it is responsible for: **Factory Method** (creating source connectors), **Prototype** (cloning transformation steps), **Template Method** (fixing the pipeline lifecycle), and **Memento** (snapshotting run state for recovery).

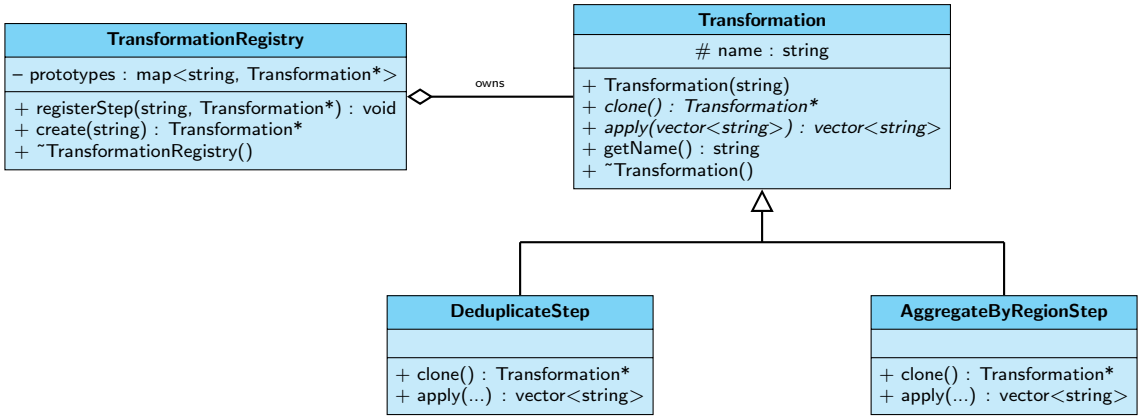
5.1 UML Class Diagram

The design has four collaborating subsystems, shown separately below to keep each diagram readable. A hollow triangle is inheritance; a hollow diamond marks an owner that must delete what it holds (free that memory in the relevant destructor); a dashed <<creates>> arrow marks an object instantiated by another. The cross-subsystem links are given by the typed attributes: a `Pipeline` holds a `ConnectorFactory*` and a `vector<Transformation*>`, and creates a `RunCheckpoint` that the `CheckpointManager` stores.

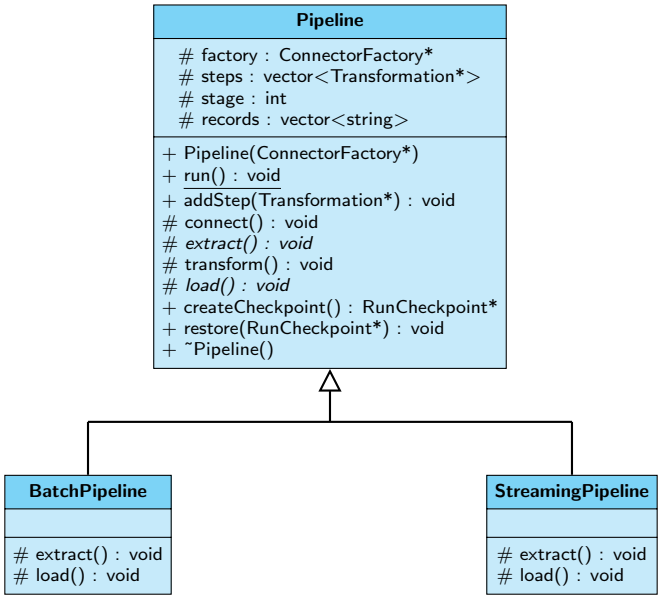
(a) Source connectors — each concrete factory creates its matching connector.



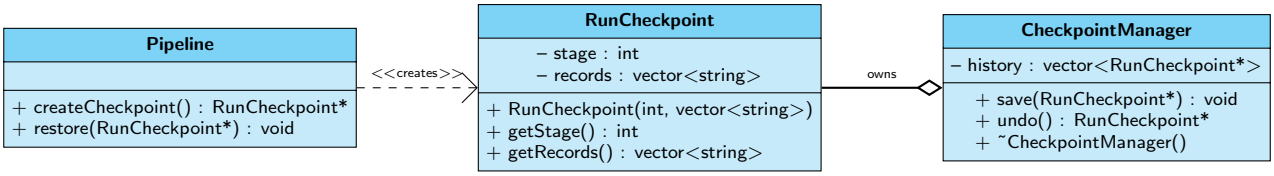
(b) Transformations — the registry owns the prototypes it clones on demand.



(c) Pipeline lifecycle — `run()` is the template method; subclasses override only `extract()` and `load()`.



(d) Checkpointing — the pipeline (Originator) creates checkpoints the manager (Caretaker) stores.



6 Assignment Instructions

Task 1: Source connectors ..... (55 marks)

New sources are onboarded often, so the engine must create the correct connector **without** the calling code naming a concrete connector class. An abstract `ConnectorFactory` (Creator) declares the factory method `createConnector()`; each concrete factory (ConcreteCreator) overrides it to return the matching concrete `Connector` (Product). Headers go in `Connector.h`/`ConnectorFactory.h` (one `.h`/`.cpp` pair per concrete class); implement in the matching `.cpp`.

`Connector` **(abstract):** constructor `Connector(string source)` stores `source`; `getSource()` returns it; `extract()` is pure virtual and returns a vector of raw record strings; the destructor should be virtual. The three concrete connectors call the base constructor with source `"postgres"`, `"restapi"`, `"csv"` respectively, and override `extract()` to return **exactly**:

- `PostgresConnector` → {`"PG:001"`,`"PG:002"`,`"PG:002"`,`"PG:003"`}
- `RestApiConnector` → {`"API:44"`,`"API:45"`,`"API:45"`}
- `CsvConnector` → {`"CSV:x1"`,`"CSV:x2"`,`"CSV:x3"`,`"CSV:x3"`}

`ConnectorFactory` **(abstract):** `createConnector()` is pure virtual; destructor virtual. Each concrete factory `new`s and returns the matching connector.

1.1 Implement the abstract `Connector` class. (8)

1.2 Implement `PostgresConnector`, `RestApiConnector` and `CsvConnector`. (12)

1.3 Implement the abstract `ConnectorFactory` class. (6)

1.4 Implement `PostgresFactory`, `RestApiFactory` and `CsvFactory`. (15)

1.5 *(PDF)* Explain why Factory Method is better here than one `createConnector(string kind)` function with an `if`/`else` chain. Refer to the open-closed principle. (6)

1.6 *(PDF)* State which Factory Method participant each of `Connector`, `CsvConnector`, `ConnectorFactory`, `CsvFactory` plays. (8)

Task 2: Transformations ..... (55 marks)

Configuring a transformation is expensive, so the engine keeps a registry of pre-configured **prototypes** and **clones** them on demand. Headers in `Transformation.h`/`TransformationRegistry.h`; implement in the matching `.cpp`.

`Transformation` **(abstract):** `Transformation(string name)` stores `name`; `getName()` returns it; `clone()` (pure virtual) returns an independent copy; `apply(vector<string>)` (pure virtual) returns the transformed records; destructor virtual. Implement two steps:

- `DeduplicateStep` (name `"dedup"`): `apply()` removes **consecutive** duplicate records, keeping the first of each run — input {`"a"`,`"a"`,`"b"`,`"a"`} → {`"a"`,`"b"`,`"a"`}; `clone()` returns a new `DeduplicateStep`.
- `AggregateByRegionStep` (name `"aggregate"`): `apply()` returns a single element `"COUNT="` followed by the record count — input of size 3 → {`"COUNT=3"`}; `clone()` returns a new `AggregateByRegionStep`.

`TransformationRegistry`**:** holds prototypes in a `map<string, Transformation*>`. `registerStep(key, prototype)` stores it, deleting any existing prototype under that key first. `create(key)` returns a `clone()` of the registered prototype, or `nullptr` if none. The destructor deletes every stored prototype (the registry owns them).

2.1 Implement the abstract `Transformation` class. (8)

2.2 Implement `DeduplicateStep` (`apply()` and `clone()`). (10)

2.3 Implement `AggregateByRegionStep` (`apply()` and `clone()`). (10)

2.4 Implement `TransformationRegistry`, with correct memory management in `registerStep` and the destructor. (15)

2.5 *(PDF)* Explain shallow versus deep copy, state which `clone()` must produce, and why the clone must be independent of the prototype. (6)

2.6 *(PDF)* Explain why the registry returns `clone()`s rather than the stored pointer, and what breaks if two pipelines shared one prototype. (6)

Task 3: The pipeline lifecycle ..... (60 marks)

Every pipeline runs the same fixed lifecycle — `connect`, `extract`, `transform`, `load` — but batch and streaming pipelines differ in *how* they extract and load. `run()` is the template method calling the stages in order; some stages are overridable primitive operations. Header in `Pipeline.h`; implement in `Pipeline.cpp`.

`Pipeline` **(abstract):**

- `Pipeline(ConnectorFactory* factory)` — stores the factory (takes ownership), sets `stage` to 0, leaves `records` empty.
- `addStep(Transformation*)` — appends to `steps` (takes ownership).

- `run()` — **the template method**; calls `connect()`, `extract()`, `transform()`, `load()` in order. **Not** virtual.
- `connect()` — shared concrete step: obtains a connector from the factory, prints `Connecting to <source><newline>` (source from `getSource()`), sets `stage` to 1, deletes the connector.
- `extract()` — pure virtual primitive; fills `records` and sets `stage` to 2.
- `transform()` — shared concrete step: applies each step in `steps` to `records` in order, replacing `records` with each step's output; sets `stage` to 3.
- `load()` — pure virtual primitive; sets `stage` to 4.
- `createCheckpoint()`, `restore(RunCheckpoint*)` — Memento hooks; declare now, implement in Task 4.
- destructor (virtual) — deletes the factory and every step in `steps`.

Concrete pipelines: `BatchPipeline::extract()` obtains a connector, sets `records` to its `extract()`, prints `Batch extract: <n> records<newline>`, sets `stage` to 2, cleans up; its `load()` prints `Batch load: <n> records written<newline>` and sets `stage` to 4. `StreamingPipeline` is identical but prints `Streaming extract: <n> records` and `Streaming load: <n> records streamed`. (`<n>` is the record count.)

3.1 Implement the `Pipeline` constructor, `addStep` and destructor with correct memory management. (12)

3.2 Implement the `run()` template method and the shared steps `connect()` and `transform()`. (10)

3.3 Implement `BatchPipeline` (`extract()` and `load()`). (12)

3.4 Implement `StreamingPipeline` (`extract()` and `load()`). (12)

3.5 *(PDF)* Explain why `run()` is deliberately not virtual, and what a subclass could break if it could override it. (6)

3.6 *(PDF)* Classify each `Pipeline` method as the template method, a concrete step, or a primitive operation. Then state, in two or three sentences, what the Factory Method and Template Method each contribute inside `Pipeline` and why they do not overlap. (8)

Task 4: Checkpointing and recovery ..... (55 marks)

Runs fail at 02:00 and must resume. The `Pipeline` is the Originator, `RunCheckpoint` the Memento, and `CheckpointManager` the Caretaker. Headers in `RunCheckpoint.h`/`CheckpointManager.h`; implement in the matching `.cpp`.

`RunCheckpoint` **(Memento):** `RunCheckpoint(int stage, vector<string> records)` stores both; `getStage()` and `getRecords()` return the captured state (the **narrow interface**). **Originator hooks on** `Pipeline`**:** `createCheckpoint()` returns a new `RunCheckpoint` holding the current `stage` and `records`; `restore(cp)` sets `stage` and `records` back to `cp`'s values. `CheckpointManager` **(Caretaker):** holds `vector<RunCheckpoint*>`; `save(cp)` appends; `undo()` removes and returns the most recent checkpoint, or `nullptr` if empty; the destructor deletes every checkpoint still held.

4.1 Implement the `RunCheckpoint` class. (10)

4.2 Implement `createCheckpoint()` and `restore()` on `Pipeline`. (10)

4.3 Implement `CheckpointManager` (`save`, `undo`, destructor) with correct memory management. (15)

4.4 *(PDF)* Explain the narrow and wide interfaces of a Memento, and which participant may use each. (6)

4.5 *(PDF)* Describe step by step how a run that fails during `load` recovers and resumes, naming `createCheckpoint`, `save`, `undo` and `restore`. (8)

4.6 *(PDF)* State which Memento participant each of `Pipeline`, `RunCheckpoint`, `CheckpointManager` plays. (6)

Task 5: Putting it all together ..... (25 marks)

Write a `main.cpp` and a `Makefile` that wire the engine together. `main()` must: (1) create a `TransformationRegistry` and register a `DeduplicateStep` under `"dedup"` and an `AggregateByRegionStep` under `"aggregate"`; (2) create a `BatchPipeline` with a `PostgresFactory`; (3) add a cloned `"dedup"` step then a cloned `"aggregate"` step, both from the registry (never constructed directly); (4) create a `CheckpointManager`; (5) call `run()`, then `createCheckpoint()` and `save()` the result; (6) delete everything you own so the program is leak-free.

5.1 Provide the `#include` directives for `main.cpp`. (6)

5.2 Provide the body of `main()` implementing steps 1–6. (13)

5.3 Provide a `Makefile` that compiles all sources with `-std=c++11` into an executable named `engine`. (6)